(c) Cadence Design Systems Inc. Do not distribute.

SystemVerilog Assertions

Course Version 5.1

Lab Manual **Revision 1.0**

cadence®

(c) Cadence Design Systems Inc. Do not distribute.

© 1990-2022 Cadence Design Systems, Inc. All rights reserved.

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright and any unauthorized use of this publication may violate copyright, trademark, and other laws. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This statement grants you permission to print one (1) hard copy of this publication subject to the following conditions:

- The publication may be used solely for personal, informational, and noncommercial purposes;
- The publication may not be modified in any way;
- Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement; and
- Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors and is supplied subject to, and may be used only by Cadence customers in accordance with, a written agreement between Cadence and the customer.

Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third-party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

(c) Cadence Design Systems Inc. Do not distribute.

Introduction to the Labs

These lab exercises are designed to make you more familiar with SystemVerilog Assertions (SVA) and the Assertion-Based Verification (ABV) methodology. They are organized in a way that takes you step-by-step through what to do first, and then slowly removes the guidance as you become more familiar with the tools and language.

Tool Specific Information

It is assumed you are familiar with the simulator and formal tools which you will be using in this course. However, there are tool tip guides to give you a quick start in the Appendices at the end of this book.

Information on managing and debugging SVA assertions with the Cadence® Jasper™ Formal Verification tool can be found in the tool guide in Appendix A at the end of this lab book.

Information on managing and debugging SVA assertions with the Cadence® Xcelium™ Simulator can be found in the tool guide in Appendix B at the end of this lab book.

Run scripts are provided and please ask the instructor to show you how to use the tool if you are unsure after first having referred to the Appendices.

Overview of Labs

The aim of the labs is to understand how to create and apply effective SystemVerilog Assertions.

The labs will use both a simulator and a formal tool. Remember that the objective of this course is to learn the SVA language. The tool we use is not important. SVA property definitions describe design behavior which is agnostic to which kind of tool is used.

The first few labs are to familiarize you with the tools that you are using and the basic steps involved in writing simple SV assertions.

Subsequent labs involve applying ABV to more complex design blocks. The goal is not to complete all the exercises during the course, but to learn from each exercise at your own pace.

© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Table of Contents

SystemVerilog Assertions

Lab 1: Simple Assertions to Verify FSM..... 5

Applying simple boolean assertions to an FSM design and exploring the difference between same and next cycle implication.

Lab 2: Sequence-Based Properties..... 11

Analyzing the evaluation of a simple sequence-based assertion in simulation. Adapting the assertion for more complex sequences and exploring the evaluation of this in simulation.

Lab 3: Auxiliary Code..... 19

Use auxiliary code, assertions, coverage, and verification components to verify a provided design, using a formal verification tool.

Lab 4: Serial Peripheral Interface Verification..... 27

Verifying transactions on an SPI bus variant.

Lab 5: Property Forming Constructs-Based Lab..... 37

Defining properties using constructs which return a property expression rather than with sequences.

Appendix A: Jasper Tool Tips Guide..... 41

Appendix B: Xcelium Simulator Tool Tips Guide..... 49

© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Lab 1: Simple Assertions to Verify FSM

(c) Cadence Design Systems Inc. Do not distribute.

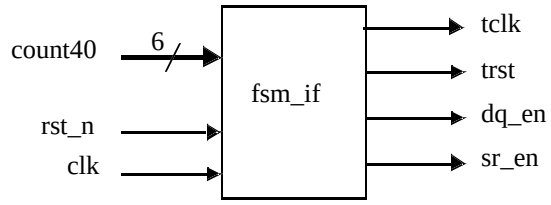
(c) Cadence Design Systems Inc. Do not distribute.

Objective

To understand the distinction between same-cycle and next-cycle implications in SystemVerilog assertions and to gain experience in using Jasper™ to verify and debug RTL designs

Scenario

The design is a simple FSM-based interface controller:

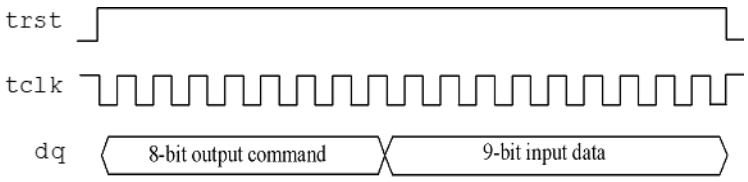


Connection Definitions

Port	Direction	Type	Description
count40	input	count value 0 to 39	Counter for serial frame
rst_n	input	single bit	master active low reset
clk	input	single bit	master clock
tclk	output	single bit	serial interface clock
trst	output	single bit	serial interface frame
dq_en	output	single bit	tri-state enable
sr_en	output	single bit	shift register enable

The FSM controls a 3-wire bidirectional serial interface to an external device. Each frame of data over the interface consists of an 8-bit serial output command followed by a 9-bit serial input response from the device.

The transaction is clocked by *tclk* and framed by logic 1 on *trst*. *trst* and *tclk* are driven directly from the FSM, whereas *dq* is driven by external logic, controlled by the FSM outputs *dq_en* and *sr_en*.

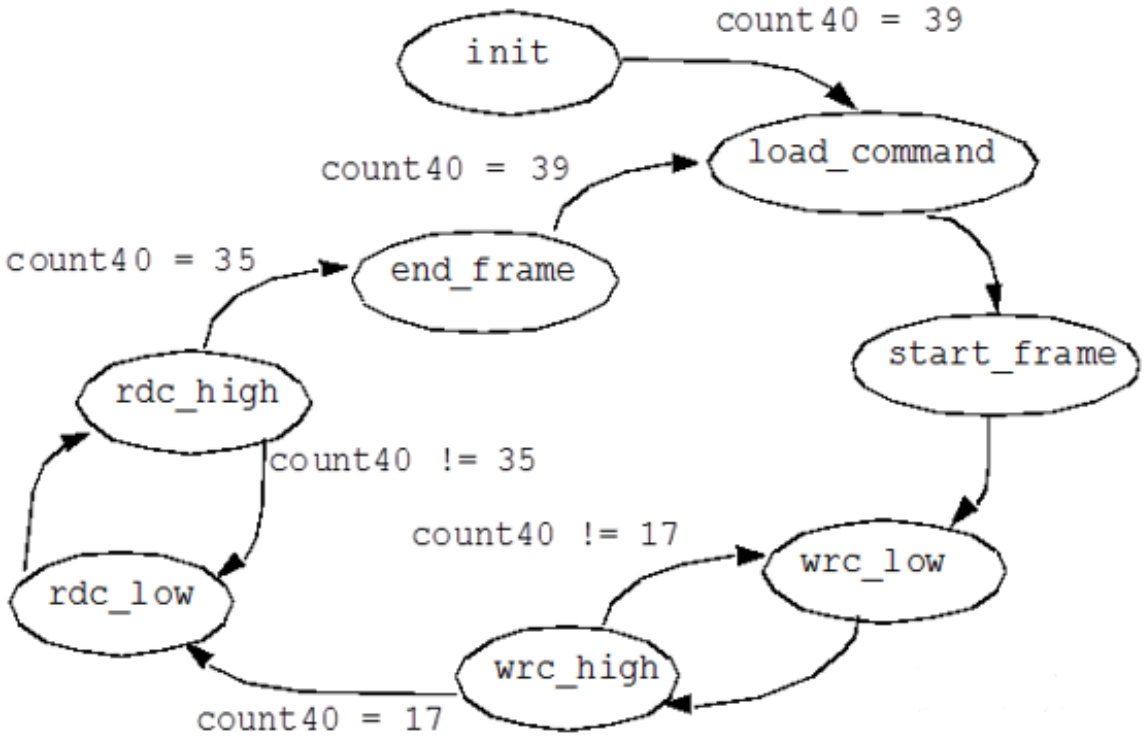


© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

State Diagram

The states and state transitions for the FSM are as shown here.



HDL FSM State Encoding	
State Name	**Value of reg state[2:0]**
init	0
load_command	1
start_frame	2
wrc_low	3
wrc_high	4
end_frame	5
rdc_low	6
rdc_high	7

© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Output Decode

The required outputs for each state are shown here.

State	Outputs {ops}			
	tclk	trst	dq_en	sr_en
init	1	0	0	0
load_command	1	0	0	0
start_frame	1	1	0	0
wrc_low	0	1	1	1
wrc_high	1	1	1	0
rdc_low	0	1	0	1
rdc_high	1	1	0	0
end_frame	1	0	0	0

Simple Boolean Assertions

1. Change your working directory to `<Labs Install Dir>/lab1_fsm`, for example: `/home/user1/SVA_X_X/lab1_fsm`.

2. A model for the FSM is provided in the `lab1_fsm/fsm_if.v` file. Note that the `fsm_if_test.v` file is a wrapper for the DUT. You don't need to edit this file. This wrapper ensures that the DUT `count40` input behaves as a counter should. Namely, it constrains the DUT `count40` input to behave like a counter.

3. Open the *`fsm_if.v`* file and familiarize yourself with the design. Do not modify any of the FSM source code at this time.

4. Write named properties to check the outputs for each state in the FSM using the same-cycle implication operator (`|->`) and assert them.

   Tip: You don't need to wait until you have finished writing all required properties before you run the tool. Initially, just write one same cycle implication property and one next cycle implication property to ensure you can define the syntax correctly. Run the tool and observe the results. Then you can use copy and paste to ensure the remaining properties start with the correct syntax.

   - The 4 outputs of the FSM have been collected into a vector called *ops* for convenience. This is how each bit of the vector relates to each output:

     ```
     ops = {tclk, trst, dq_en, sr_en};
     ```

© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

- The design uses a SystemVerilog *localparam* type to define the state variable, so we use the state name when checking the state variable, such as:

  ```
  (state == start_frame)
  ```

- The assertions are very similar to each other. Use copy and paste when creating these and the following assertions!

5. Write named properties to check the conditional state transitions of the FSM using the next-cycle implication operator (|=>) and assert them.

6. Compile the FSM and correct any bugs in the assertions.

Testing the Boolean Assertions

1. A formal testbench for the FSM is supplied in the *lab1_fsm/fsm_if_test.v* file in order to correctly constrain the design count input to act as a counter would be expected in the real design.

2. Compile and elaborate by running the *./RUNME.sh* script. Run the proofs using the Tcl command `prove -all` and check your assertion results. If you are not familiar with the Jasper tool, then refer to the appendix, which describes the basic operations you will need to perform.

3. There are two deliberate errors in the FSM implementation compared to the design specification. Your assertions should detect and report these errors!

4. Fix the errors in the FSM design. Rerun the tool to check the errors have been corrected.



© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Lab 2: Sequence-Based Properties

(c) Cadence Design Systems Inc. Do not distribute.

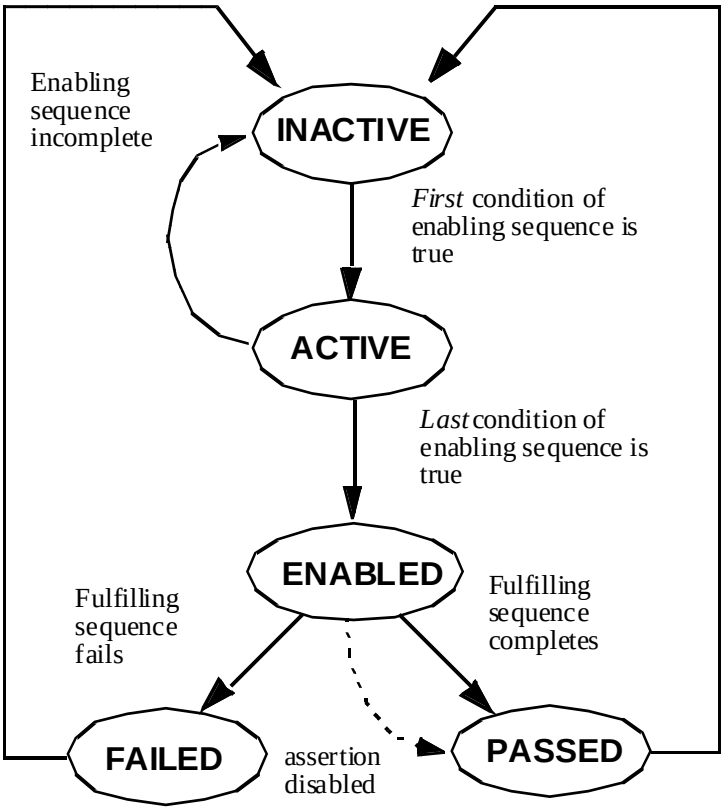
(c) Cadence Design Systems Inc. Do not distribute.

Goals

In this lab, you create a simple and a complex sequence-based assertion and simulate them from a supplied testbench to explore how your simulator handles and reports the various states in a multicycle assertion evaluation.

Scenario

A multicycle sequence-based assertion can be in any one of several states:



State	Description
INACTIVE	Waiting for enabling sequence to begin
ACTIVE	Enabling sequence started and being evaluated
ENABLED	Enabling sequence completed Evaluating fulfilling sequence
PASSED	Fulfilling condition completed
FAILED	Fulfilling condition failed

© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Simulating the Simple Sequence Assertion

In this lab, you add your assertions to the supplied testbench.

1. Change your working directory to `<Labs Install Dir>/lab2_sequence`, for example: `/home/user1/SVA_X_X/lab2_sequence`.

2. Navigate to the `lab2_sequence` directory and open the *seqtest.v* file.

3. Create and assert a property *SIMPLE_SEQ*, clocked off the negative edge of *CLK*, to the following specification:
   - The enabling sequence is *A* at logic 1 followed immediately by *B* at logic 1 followed immediately by *C* at logic 1 (in consecutive samples).
   - The fulfilling sequence follows in the cycle after the enabling sequence completes and is *J* at logic 1 followed by *K* at logic 1 in consecutive samples.
   - The property is disabled if *X* is at logic 1 for any cycle.

4. Examine the rest of the testbench and make sure you understand its operation.

   Note that for each subtest, the testbench passes a character string to a *do_test* subprogram. The **;** character is the cycle delimiter. Other characters correspond to signals driven high by the subprogram during that clock cycle.

   Leave the commented-out section. This section is for testing the complex sequence.

5. Compile *seqtest.v* using the run script *./RUNME.sh* and fix any compilation errors reported. For this lab, we will be using the Xcelium™ simulator.

6. Set a breakpoint at the line which contains *breakpoint=breakpoint+1* in the *do_test* task.

   You can use this breakpoint to step through the subtests. The simulator will otherwise stop at only the failing tests.

7. If you have successfully created the simple sequence assertion, you should see three assertion failures when you simulate the design. Two of the failures occur within the same subtest:

   a. Run the simulation up to the first failing subtest.

   Question 1 – Which subtest failed and why did it fail?

   Answer: _____

   Tip: Use the waveform viewer to examine the signal in the sequence and the states of the assertion. Compare this failing test to previous passing tests.

© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

b. Run the simulation up to the second failing subtest.

Question 2 – Which subtest failed and why did it fail?

Answer: _____



© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Simulating the Complex Sequence Assertion

In this lab, you add and simulate complex sequence assertions up to the failing test.

1. Add a complex sequence assertion *COMPLEX_SEQ*, clocked off the negative edge of *CLK*, to the following specification:
   - The enabling sequence is C at logic 1 followed immediately by one to three consecutive samples, in which B is at logic 1 followed immediately by A at logic 1.
   - The fulfilling sequence follows in the cycle after the enabling sequence completes and is J at logic 1 for four samples (which may be nonconsecutive), the last of which is followed immediately in the next sample by K at logic 1.
   - The assertion is disabled if *X* is at logic 1 for any cycle.

2. Comment out the stimulus for the simple sequence assertion and uncomment the stimulus for the complex assertion.

3. Compile *seqtest.v*, fix any compilation errors, and load into the simulator.

4. (Optional) You can set a breakpoint on the signal *breakpoint* to step through the subtests.

   The simulator otherwise stops at only the failing subtests.

5. If you have successfully created the complex sequence assertion, you should see one failure when you simulate the design:

6. Run the simulation up to the failing test.

   Question 3 – Which subtest failed and why did it fail?

   Answer: _____

© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Answers

Question 1 – Which test failed and why did it fail?

Answer: The third subtest of the simple assertion fails to complete the fulfilling sequence – J is not immediately followed by K.

Question 2 – Which test failed and why did it fail?

Answer: The 5th subtest of the simple assertion fails to complete the fulfilling sequence for two overlapping traces of the sequence. Both traces fail because K does not immediately follow J. These two failures occur in consecutive cycles.

Question 3 – Which test failed and why did it fail?

Answer: The 6th subtest of the complex assertion fails to complete the fulfilling sequence – K does not immediately follow the fourth occurrence of J.



© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

(c) Cadence Design Systems Inc. Do not distribute.

Lab 3: Auxiliary Code

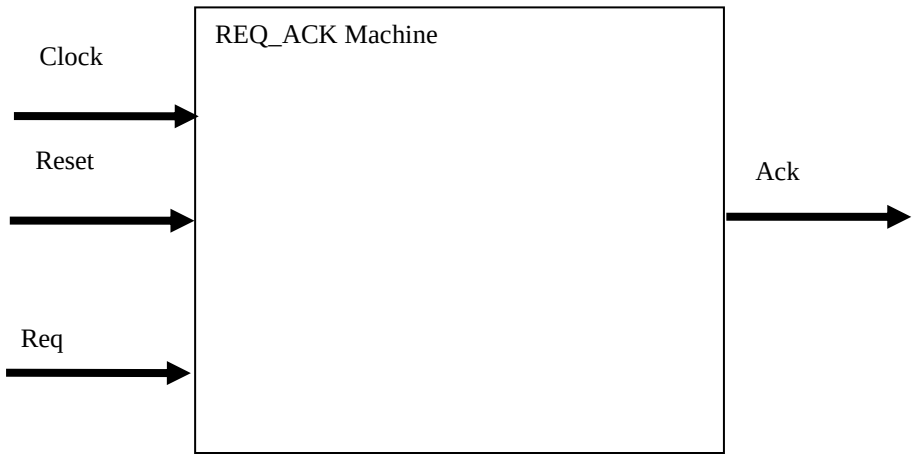
(c) Cadence Design Systems Inc. Do not distribute.

(c) Cadence Design Systems Inc. Do not distribute.

Objectives

- Show how auxiliary code can be used to address problems that assertion languages cannot deal with.
- Show how we can define behavior using more than one property instead of trying to define all behavior in a single property.
- Observe issues related to defining checks which have no time bound, i.e., they define behavior to infinity.

REQ_ACK Design Details



The drawing above represents the block that we will verify during the lab.

As far as we are concerned, we can treat the design as a black box.

We do not necessarily need to be concerned with the exact details of what happens inside. All that we want to verify is that we observe the correct behavior at the interfaces of the block.

The requirement for the behavior of this block is as follows:

- A valid request is when req is high at a rising edge of clock.
- A valid acknowledgment is when ack is high at a rising edge of clock.
- Every req should eventually have an associated ack.
- There should not be an ack without an associated req.
- The ack can occur anytime.

© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

- There does not need to be ack in response to a req, before the next req.
- There can only be a maximum number of 4 reqs which have not had an ack at any given cycle.
- A new request cannot be acknowledged in the same cycle in which it is made.
- An ack for a *previous* request may occur in the same cycle as a new request

Reviewing the RTL Design

1. Change your working directory to:

   ```
   <Labs Install Dir>/lab3_auxcode/jg
   ```

   For example:

   ```
   /home/user1/SVA_X_X/lab3_auxcode/jg
   ```

2. In an editor open the file `../rtl/reqack_duv.v`

3. Notice that the file contains RTL design code. Later on, you can read the contents of the file in greater detail when we run a proof that takes a long time. For now, we will just look quickly at the RTL.

4. The design uses a counter to generate a count sequence in a register. The RTL design code looks at specific elements in this register to determine whether ack should be driven or not. The effect that this has is that `req`s are `ack`ed a [pseudo]random number of cycles later. From just looking at the RTL code it is hard to predict whether the design works under all conditions. Simulation would probably require many millions of cycles to test every scenario.

5. In this simulation, it would be hard to be confident that you have simulated all circumstances under which the design could fail.

6. This is what formal excels at. If there is a way for the design to fail, then formal will find it. We will be using Jasper™ for this lab.

© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Implementing the Verification Code

Note: These labs do not include step-by-step instructions and do not tell you exactly what you need to type. You will probably initially feel daunted by the challenge but take things one step at a time, refer to the information in the lecture slides and ask the instructor if you get stuck.

1. In an editor open the file `../rtl/reqack_vcomp.v`.

2. Notice that the file contains an empty module with only the ports defined. This is a verification component or Vcomp. It should only contain code related to verification.

3. Add to the verification component any auxiliary code, assertions, and coverage statements that you think will be needed to verify the behavior required as defined at the beginning of this lab.

4. It is completely your decision how to implement the required verification code, but here are some hints that may help you.

5. You cannot verify the design with just SVA properties.

6. You will need more than 1 property to check the DUT behavior is correct.

7. The signals defined in the port list of the Vcomp are the only DUT signals that you need in order to determine the correct behavior of the DUT.

8. We need coverage to check for a “sane” verification environment. We define coverage to state the kinds of things that our design and auxiliary code should be capable of doing. Remember that in formal, coverage asks the question, “Is it possible for this scenario to occur?”. In simulation, the same SVA coverage code asks the question, “Did this scenario occur during this simulation run?”

9. Write an assertion to check specifically that if we observe a `req`, then we eventually get an `ack`. Namely, write a liveness property. This might fail when you are not expecting it to. We will discover that later on.

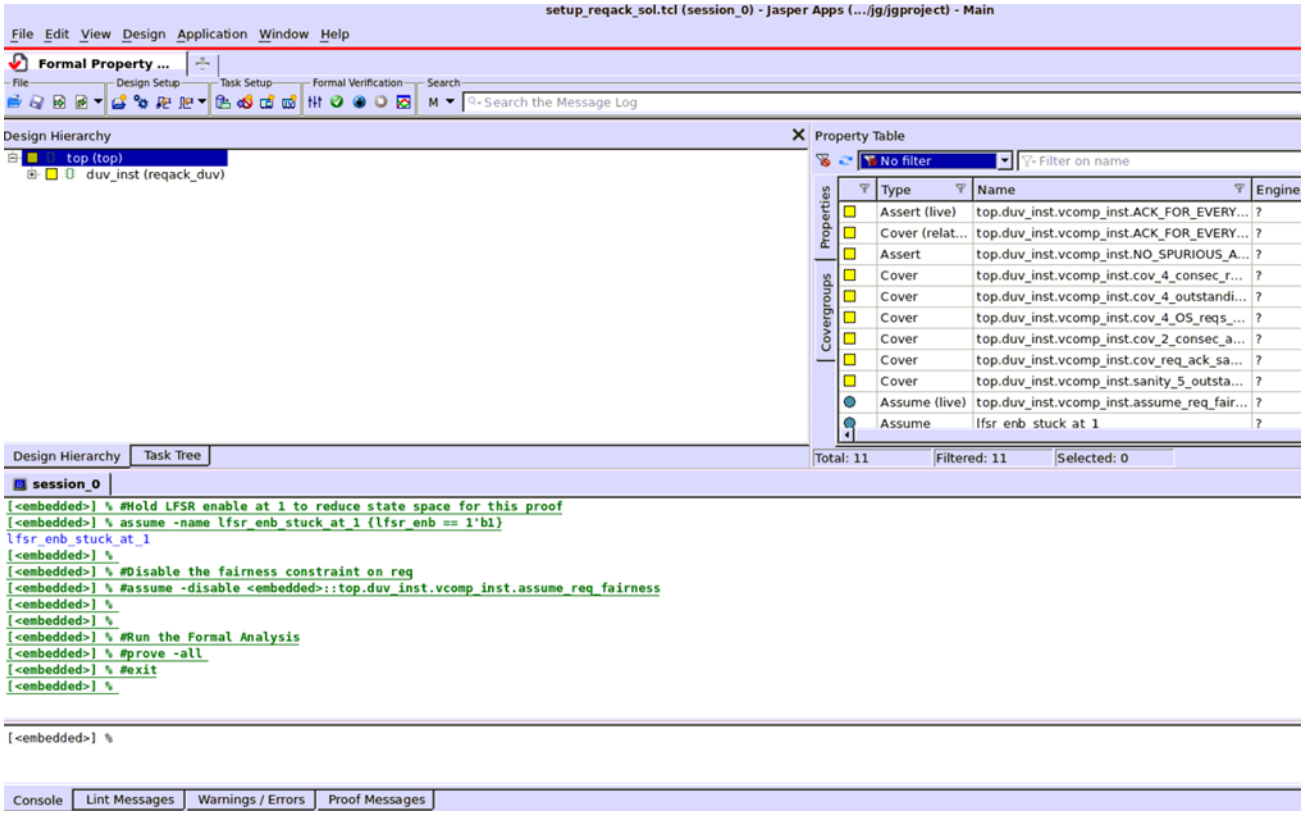
10. There are no statements about how the DUT inputs should behave; therefore, you should not define any assumptions

© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Running the Proof

1. Run the proof by running the script `./RUNME.sh`. This script uses the `.f` and `.tcl` files. You may be interested in looking at those files and the script file to see how your verification component gets “attached” to the DUT.



2. In the property table, you see the user-defined assertions (Assert) and constraints (Assume). Cover are the coverage statements. In formal, cover statements ask the question, “Is it possible for this to happen”. This is not the same as an assertion. An assertion says, “Is this property true under all circumstances.”

3. All of your assertions and cover statements should pass. Keep reading if you see one of your checks unexpectedly fail, as we previously hinted.

4. You probably wrote a property that checks that, for infinity, every request gets an acknowledgment, eventually. For example, the `ACK_FOR_EVERY_REQ` property in the screenshot above will be shown to fail.

5. If you wrote such a property, then look at the counter example (CEX). The CEX shows one way in which the property can fail. To launch the property CEX, **double**-click on the property name in the property table. Does the CEX seem to indicate any bad behavior, as defined at the beginning of this lab?

© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Understanding Fairness Constraints

1. What has happened here is the formal tool has driven the `req` input in an "unfair" way. What your property probably says is that "if there are no outstanding requests and then we have a request, then we **must** reach a state where there are no outstanding requests again." All Jasper has to do to demonstrate failure and to make one request and then carry on making more requests for infinity. The design will not be able to acknowledge quickly enough to ever acknowledge all outstanding requests.

2. What we really want is to know whether it is **always possible** to get back to the state where there are no outstanding requests. This is different from what your version of the `ACK_FOR_EVERY_REQ` property probably says. That property says we **must** get back to a state where there are no outstanding requests again.

3. The "always possible" check described in the sentence above is called a `no deadlock` check.

4. There is no way to express this requirement in SVA. PSL has a way of expressing this, and it is called the Optional Branching Extensions (OBE). OBE describes what is called Computation Tree Logic (CTL). There are no commercial tools that support CTL. SVA only allows you to describe Linear-Time Temporal Logic (LTL), i.e., things which can also be simulated.

5. In order to perform the check that we want to do, we need to define something called a `fairness` constraint. This forces Jasper to drive the `req` input in a "fair" way. The constraint `assume_req_fairness` already defined for you in the Vcomp, says that at some point in the future, `req` must be logic 0 for 9 cycles. This is long enough for all outstanding requests to be acknowledged. This means that the formal tool cannot show us a CEX by continually making requests for infinity.

6. Close the formal tool using `File->Exit` from the Jasper GUI menu.

7. In order to allow Jasper to have visibility of this fairness constraint already defined for you in the file `../rtl/reqack_vcomp.v`, comment the line as shown below in the file `setup_reqack.tcl`:

```
#Disable the fairness constraint on req
assume -disable <embedded>::top.duv_inst.vcomp_inst.assume_req_fairness
```

8. Execute the script `./RUNME.sh` again.

9. Notice that after a couple of seconds all covers and assertions pass except for your version of the property `ACK_FOR_EVERY_REQ`.

© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

10. While the proof is running you can read the file `reqack_duv.v` in more detail. Ask the presenter if you have more questions.

11. Hopefully, by now, you can understand that the issues in using SVA are much more difficult than the simplicity of the syntax might have you believe.



© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Lab 4: Serial Peripheral Interface Verification

(c) Cadence Design Systems Inc. Do not distribute.

(c) Cadence Design Systems Inc. Do not distribute.

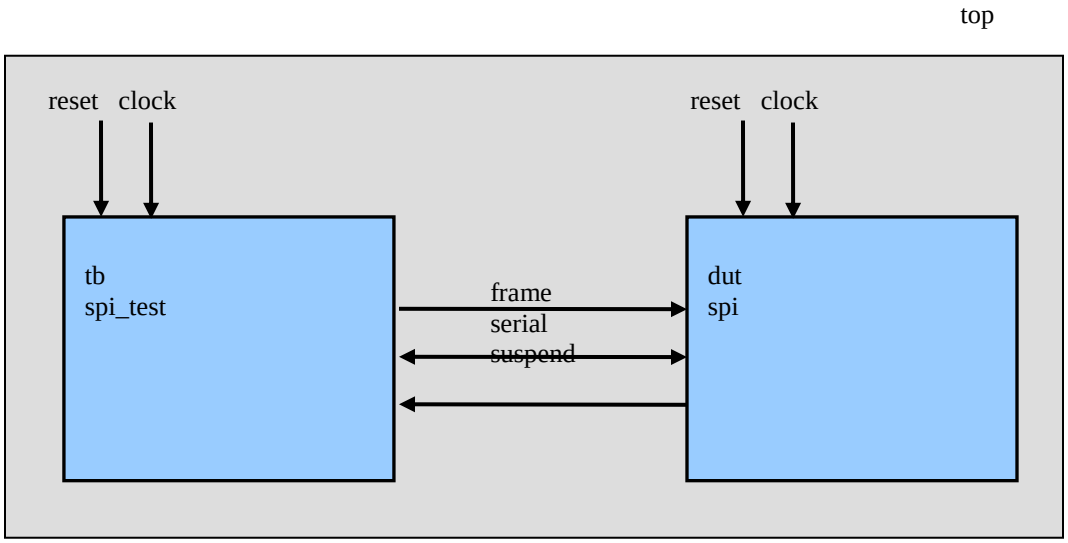
Serial Peripheral Interface (SPI) Verification

In this lab, you write assertions to verify that SPI transactions are complete in a legal way. To gain experience in assertion variables and defining complex clocking expressions. To write coverage statements to check how many legal and illegal transactions were observed.

Lab Overview

This lab involves the verification of a simple variant of an SPI (Serial Peripheral Interface) using a simulator.

High-Level Diagram: SPI Hierarchy



Where:

- **frame** is the framing bit, going *high* at the beginning of the transaction and *low* at the completion.
- **serial** is the *bidirectional* data bit, valid on the rising edge of clock.
- **suspend** is used by the **dut** to temporarily suspend the current transaction when *high*, (more details later).
- **clock** and **reset** are control signals from the **top** module. **reset** is active high and asynchronous.

© 2022 Cadence Design Systems, Inc. All rights reserved.

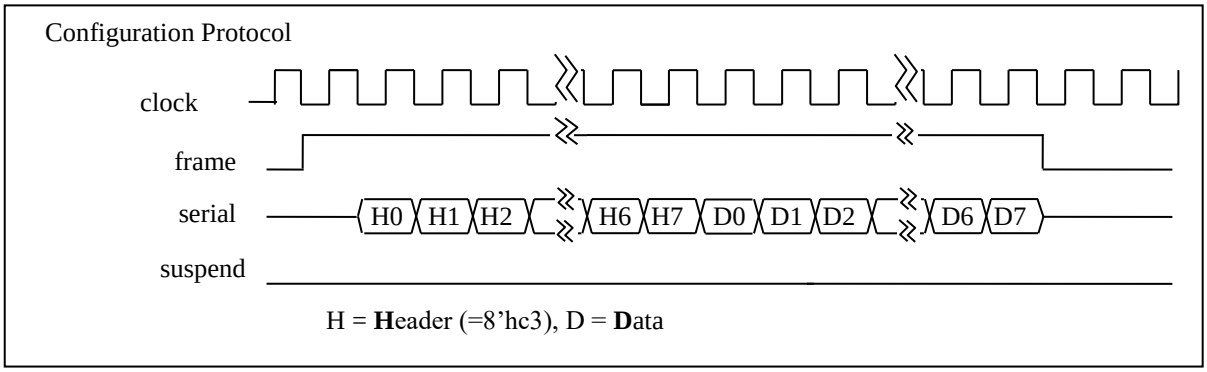
(c) Cadence Design Systems Inc. Do not distribute.

Transaction Protocol

All signals are active high and are driven on the **falling** edge of the clock. Only three transactions are supported by this simple SPI variant.

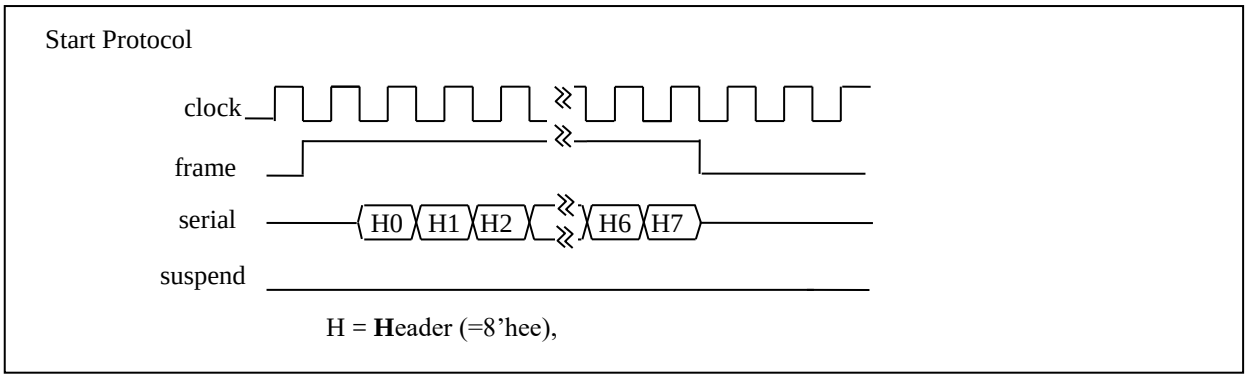
Configuration Transaction

Sets the configuration registers of the device. The *frame* signal is asserted and from the next clock, the 8-bit configuration header (`8'hc3`) is sent over *serial* to the device, LSB first. After the header, the 8-bit config data is transmitted over *serial*, to the device, from LSB to MSB. On the next clock after bit 7 of the config data, *frame* is de-asserted and the transaction is complete. Assume for the moment suspend remains low.



Start Transaction

Initializes device operation. The *frame* signal is asserted and from the next clock, the 8-bit start header (`8'hee`) is sent over *serial*, to the device, LSB first. On the next clock after bit 7 of the start header, *frame* is de-asserted and the transaction is complete. Assume for the moment suspend remains low.

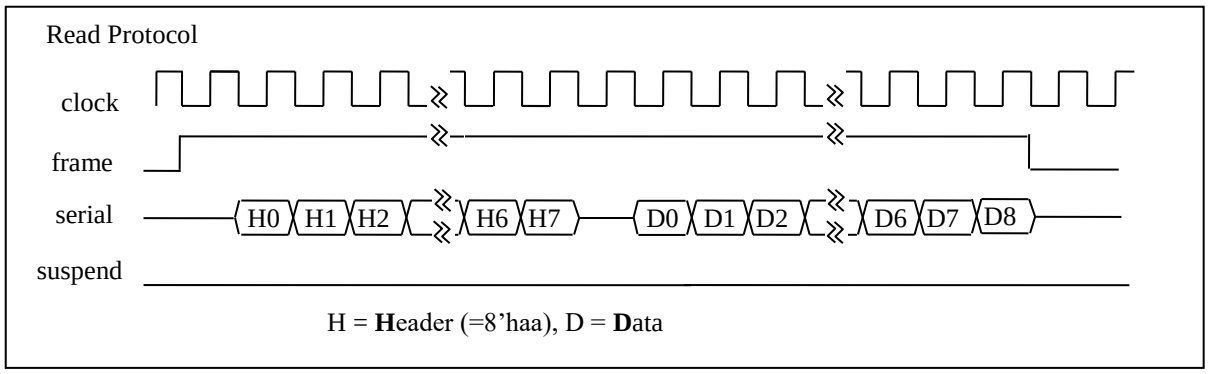


© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Read Transaction

Reads a single byte of data from the device. The *frame* signal is asserted and from the next clock, the 8-bit read header (`8'haa`) is sent over *serial*, to the device, LSB first. After the header, there is a single turnaround cycle where *serial* is ignored. Then 9-bit read data is transmitted over *serial*, from the device, from LSB to MSB. On the next rising clock after bit 8 of the read data, *frame* is de-asserted and the transaction is complete. Assume for the moment suspend remains low.



Creating SPI Assertions

1. Change your working directory to:

   ```
   <Labs Install Dir>/lab4_spi
   ```

   For example:

   ```
   /home/user1/SVA_X_X/lab4_spi
   ```

   Note: These labs do not include step-by-step instructions, and do not tell you exactly what you need to type. You will probably initially feel daunted by the challenge, but take things one step at a time, refer to the information in the lecture slides, and ask the instructor if you get stuck. This is quite a difficult real-world problem.

   It is completely up to you what to implement. The end goal is that your assertions pass when expected and fail when expected during the predefined tests that will be provided to you. However, here are some tips that can help you:

- You can recognize each part of the protocol as a sequence, such as the start of the transaction, transfer of header, and completion of transfer.
- There are only three legal ways (maybe you can call them sequences) to complete a transaction once the header has been received.
- The requirement for something to occur, like a requirement for a fulfilling condition, is once the frame has been seen to rise.

© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

- You do not need to indicate an assertion failure as soon as it has been observed. For example, if an invalid header was observed, you don't necessarily need to indicate a failure at the earliest possible cycle because it may make the property harder to write. You can wait until the end of the transaction or after some number of cycles instead.
- Local variables are very useful for collecting serial bits into a vector, i.e., deserialization. Remember we had a slide that discussed this.

Creating a Configuration Assertion

1. Review the Configuration Transaction specification. Write assertion(s) where indicated in the comments, in the *top.sv* file, to check that this transaction completes correctly. Note the following:

   a. Use the clock *rising* edge to evaluate your properties. You do not want to consider scheduling issues for this lab.

   b. The enabling condition for this transaction assertion will be a *rise* of *frame*. Note that *rise* means rise relative to the property evaluation clock, not the rise of the signal *frame*. Declare a separate sequence for this and as many of the other conditions as possible. Remember, divide and conquer complex sequences into simpler sequences which can then be used as building blocks for describing the more complex sequences.

   c. You need to capture the header for each transaction for identification and verification. The values for each header are defined for you in the *header_def.inc* file. Use one or more assertion variables to capture the config header data.

      Hint: You saw a useful example in the slides of collecting serial data using assertion variables.

   d. The config data itself will not be checked, but your assertion must verify that exactly 8 bits of data are sent before a falling edge on *frame* completes the transaction. This is common with assertions. In general, we write assertions about control functionality rather than data path functionality.

2. Check your assertion in simulation.

   Compile the *spi.sv*, *spi_test.sv* and *top.sv* files using the script provided to run simulation. The script is called *./RUNME.sh*.

   a. When the simulator command console appears, at the simulator prompt, type in the transaction number to send a configuration transaction, for example:

      ```
      xmsim> run; deposit txno = 1; run
      ```

   You will see a menu printed in the simulator console showing you the options for different values of *txno*.

© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

If you are not familiar with the simulator and you wish to view waveforms or look at assertions in the assertion browser, then look in the tools tips guide, which is located immediately following this lab.

If you want to return to 0ns in simulation time to perform another transaction, then use the following commands:

```
xmsim > reset; run; deposit txno = 3; run
```

b. Check your assertion is working correctly by observing the waveform and the output of xmsim Tcl command `assert -summary`.

Start Transaction

1. Review the Start Transaction specification. Modify your existing assertion(s) to check and verify start transactions as well. Specifically, if the header is 8'hee, then the transaction immediately completes with frame de-asserted.

2. Check your assertion in simulation by typing in the transaction number for a start transaction.

3. Ensure that your assertion also still works with Configuration Transactions.

Read Transaction

1. Review the Read Transaction specification and modify your existing assertion(s) to check and verify read transactions as well. Specifically, if the header is *8'haa*, then exactly 9 bits of data are sent before a falling edge on *frame* completes the transaction. Remember the extra turnaround cycle between header and data.

2. Check your assertion in simulation by typing in the transaction number for a read transaction.

3. Ensure that your assertion also still works with Configuration and Start Transactions.

Illegal Transactions

1. After your assertion is working with legal transactions, use the prompt options to send illegal transactions and make sure your assertion(s) detects them.

© 2022 Cadence Design Systems, Inc. All rights reserved.

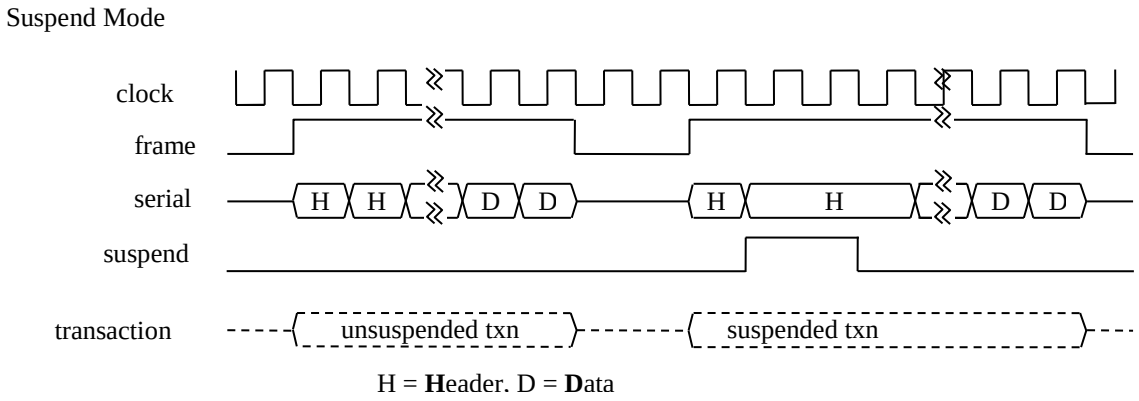
(c) Cadence Design Systems Inc. Do not distribute.

Dealing with Suspend

In this lab, you modify assertions to handle transaction suspension from the device.

The device has the ability to assert the *suspend* signal to suspend the current transaction. *suspend* can be asserted any time after the rising edge of *frame*. While in suspend mode, the current bit of the transaction on *serial* cannot change.

Suspend can only be asserted for a **maximum** of **3** clock cycles at a time but can be asserted multiple times inside the same transaction.



Suspend Mode Assertion

1. Modify and add extra assertion(s) to handle suspend mode.

   Tip: Hint – remember you can create an additional signal expression for the clocking expression of an assertion using the `iff` clause

2. Check that your assertion is working correctly in simulation.

   a. Compile and run your simulation. Allow the testbench to randomly apply the suspend signal this time. At the simulator prompt, enter the following commands. You can try each *txno* value for all of the different kinds of transactions, for example:

   ```
   xmsim> run; deposit top.dut.suspend_on = 1; deposit txno = 1; run
   ```

3. Write another assertion to check that *suspend* is never active for more than 3 cycles. Remember, you don't need to specify all checks in one assertion as it makes it harder to write and understand.



© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Adding Functional Coverage

In this lab, you add coverage to the SPI transactions.

We need to track the number of each transaction that is sent to the SPI device using SVA coverage features.

Coverage

1. Declare coverage statements to count the number of completed Configuration, Start and Read transactions sent to the device.
   - Good design practice dictates that we reuse as many existing sequence declarations as possible. Try to avoid duplicating any transaction functionality in both assertions and coverage – declare additional sequences as necessary.

2. Compile and run your simulation. Use the simulator options to send different numbers of transactions to the device and make sure your coverage correctly counts the transactions.

3. Add coverage to count the number of transactions sent with unrecognized headers.



© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

(c) Cadence Design Systems Inc. Do not distribute.

Lab 5: Property Forming Constructs-Based Lab

(Optional Extra Credit Depending on Time)

(c) Cadence Design Systems Inc. Do not distribute.

(c) Cadence Design Systems Inc. Do not distribute.

Objective

To understand and make use of the property forming constructs which we have covered in the lectures and to create some properties which we could not necessarily have implemented using sequences alone.

1. Change your working directory to:

   `<Labs Install Dir>/ lab5_PropFormingConstructs`

   for example:

   `/home/user1/SVA_X_X/lab5_PropFormingConstructs`

2. Open the *propFCtest.v* file and add an assertion that performs the check described below. Do not use sequences to implement your property for the following reasons:

   - The purpose of the lab is to use property-forming constructs.
   - You won't be able to describe the check just with sequences.

   To reiterate: do not use sequences to describe this property. If you use the cycle delay operator `##N`, then you are doing it wrong.

   Super Extra Credit

   Optionally and if you are interested, otherwise bypass this paragraph and go to the next step. When using new SVA operators and constructs in properties, it can be hard to understand when they pass or fail. Using Jasper™ (Formal tools) is the best way to see what your property does without having to think about what stimulus to apply or worry that you missed some corner case. Open ***another*** Linux terminal and `cd` to:

   `<Labs Install Dir>/SVA_X_X/SVA_Test_Env_formal_and_sim/jg_equiv`

   In file `chk_equiv.v` enter your assertion and comment out all other properties. Run the `RUNME.sh` script in that directory. This will open Jasper and you will see your assertion failing and the cover (witness) passing. There is no design, so the wires are only driven by the tool to show you a CEX and a witness trace, i.e., the shortest example of a pass and a fail. The WaveEdit feature can be used to modify the waveform you see to learn more about the property. Ask your instructor to show you how to use WaveEdit if you are interested.

3. This is the definition of the assertion required. If B is true, then from the next cycle D must become true before C becomes true. In the cycle after B we can wait any number of cycles for D. However, C cannot be true in the same cycle as D becomes true. This is known as "D strictly before C" in assertion languages Use the posedge of the clock to evaluate the property.

4. When you think you have defined the property correctly in file *propFCtest.v* then run the script `RUNME.sh` in the directory `<Labs Install Dir>/ lab5_PropFormingConstructs` to run the Xcelium simulator. If you have written your properties correctly, then these are the results you should obtain:

© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

a. Fail at 75 ns
b. Fail at 175 ns
c. Pass at 295 ns

5. Open the *propFCtest.v* file and add another assertion that checks performs the check described below. Do not use sequences to implement your property for the following reasons:
   - The purpose of the lab is to use property-forming constructs
   - You could describe the check just with sequences, but that would be missing the point of the lab. 😉

   To reiterate: do not use sequences to describe this property. If you use the cycle delay operator `##N`, then you are doing it wrong.

6. This is the definition of the assertion required. If A is true, then from the next cycle E must be true forever, i.e., at every future cycle. Use the posedge of the clock to evaluate the property.

7. When you think you have defined the property correctly in file *propFCtest.v* then run the script `RUNME.sh` in the directory `<Labs Install Dir>/lab5_PropFormingConstructs` to run the Xcelium simulator. If you have written your properties correctly, then these are the results you should obtain

   d. Fail at 435 ns
   e. Fail at 595 ns
   f. Pass at the end of simulation (795 ns)

   Question 1 – Why is the assertion only shown as passing at the end of simulation?

   Answer: _____

Answers

Question 1 – Why is the assertion not shown as passing until the end of simulation?

Answer: Because always means “forever” and in simulator forever means the end of simulation



© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Appendix A: Jasper Tool Tips Guide

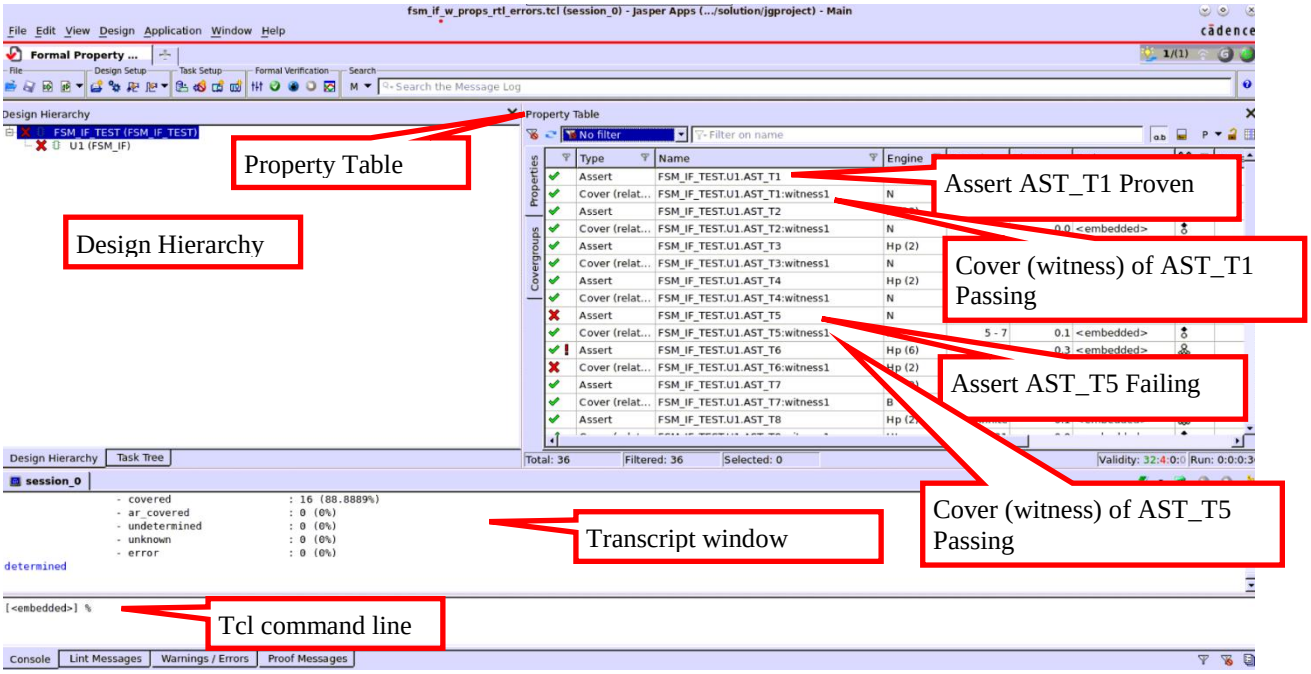
(c) Cadence Design Systems Inc. Do not distribute.

(c) Cadence Design Systems Inc. Do not distribute.

Introduction

This appendix is intended to give you a quick start to using Jasper™ to analyze and debug assertions and also to observe covers. The run scripts provided in the lab exercises will do all of the necessary tool setup and compilation tasks for you. Jasper is an intuitive tool. Most of the time, if you hover your mouse over things, then it tells you what they are. If you select something and then click the right-hand mouse button, then a context-sensitive menu will make it clear what you can do from there.

Main Jasper Window



1. You can navigate the design hierarchy and the property table will show the properties at the level of hierarchy you have selected.

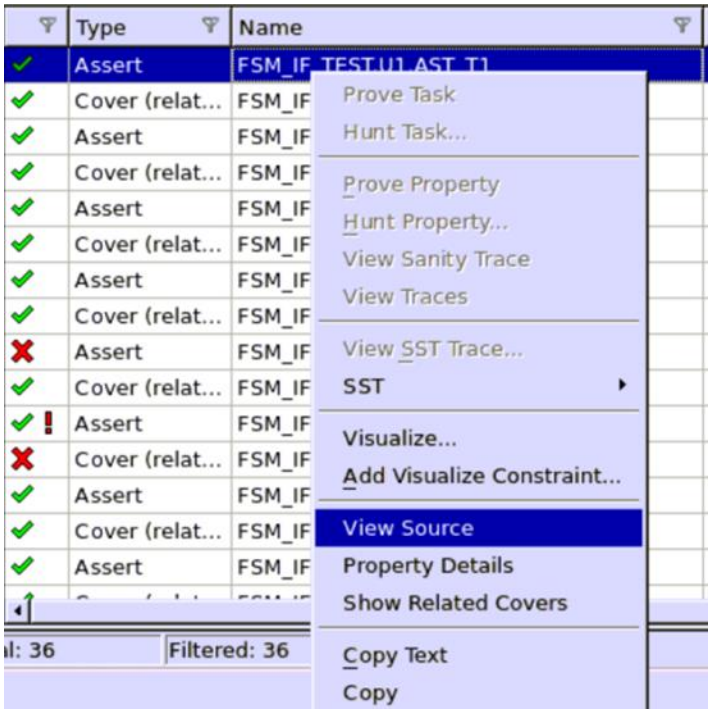
2. The property table shows the results of the proof. To run the proofs either enter the Tcl command `prove -all` (it's likely that your run script already does that). Alternatively, click the green button with the white tick in the Formal Verification toolbar  .

3. The property table can be sorted by any of the columns. If you sort by name, then you will see the unique hierarchical name of your property. You will also see a related cover (called a witness, an example of the property completing). Just because a property can be shown completing once does not mean that it is always true (proven).

© 2022 Cadence Design Systems, Inc. All rights reserved.

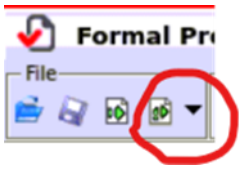
(c) Cadence Design Systems Inc. Do not distribute.

To see a property's source code, select the property, click the RMB, and select **View Source**.



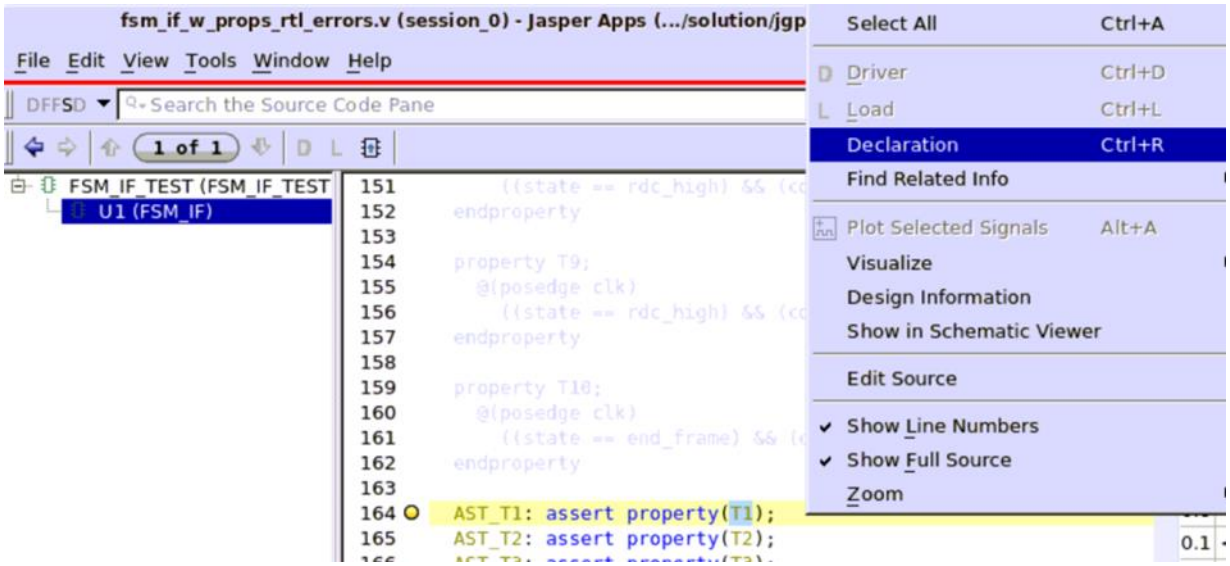
4. If the source code line is an instantiation of a previously declared property, then select that line of source code, click the RMB and select **Declaration** to see the property declaration.

5. If you have debugged a problem and edited the source code, then you can do all of the recompilation and rerunning of proofs without closing the GUI and giving up your LSF job. In order to do this, click the 4th button from the left in the File toolbar



© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.



Interpreting Property Table Results

✔	Assert	FSM_IF_TEST.U1.AST_T4
✘	Assert	FSM_IF_TEST.U1.AST_T5
✔ !	Assert	FSM_IF_TEST.U1.AST_T6
✔	Assert	FSM_IF_TEST.U1.AST_T7

1. Referring to the table above, a green tick for an assert means that the property is proven under all circumstances. There is no waveform to observe in this case. For example, like `AST_T4` above. We could, however, look at the witness cover for `AST_T4`, which will be named `AST_T4:witness1`. **Double**-clicking `AST_T4:witness1` will show us an example waveform of the property passing.

2. A red cross means that the assertion fails. **Double**-clicking the property will open a Visualize window. This is how we debug all properties. In this case, Visualize will show us an example waveform of the property failing. This waveform is known as a counter-example. We debug that waveform just as if the waveform came from a simulation. For example, like `AST_T5` above.

3. A green tick with a red exclamation mark next to it is a harder to understand result. It means the assertion is proven (namely, there is no counter example). However, the red exclamation mark tells us that there is a failing related cover. What this often means is that for a property with implication, e.g., `A |=> B`, is that A can never occur; hence, we never actually check for B. This is known as a vacuous pass and is something that we need to fix. For example, like `AST_T6` above.

© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

✗	Assert	FSM_IF_TEST.U1.AST_T5
✓	Cover (related)	FSM_IF_TEST.U1.AST_T5:witness1
✓ !	Assert	FSM_IF_TEST.U1.AST_T6
✗	Cover (related)	FSM_IF_TEST.U1.AST_T6:witness1

4. Referring to the table above, we can see that there is a counter example for `AST_T5`, i.e., it doesn't *always* pass. If we look at the related cover `AST_T5:witness1` then we can see that there is at least one way in which it can pass. If we **double**-click `AST_T5` then we see the counter example (CEX) shown as a waveform in Visualize. If we **double**-click `AST_T5:witness1`, then we see a cover of the property passing shown as a waveform in Visualize.

5. For `AST_T6`, we can see that the property is proven, i.e., no CEX, but the related cover `AST_T6:witness1` is unreachable. Unreachable means that the cover can never occur; hence, there is no waveform to show. You have to debug why `AST_T6:witness1` is unreachable. The tool can tell you that the cover is unreachable but clearly cannot tell you why because that is a result of user error in code or in tool setup.

Debugging CEXs and Observing Covers in Visualize

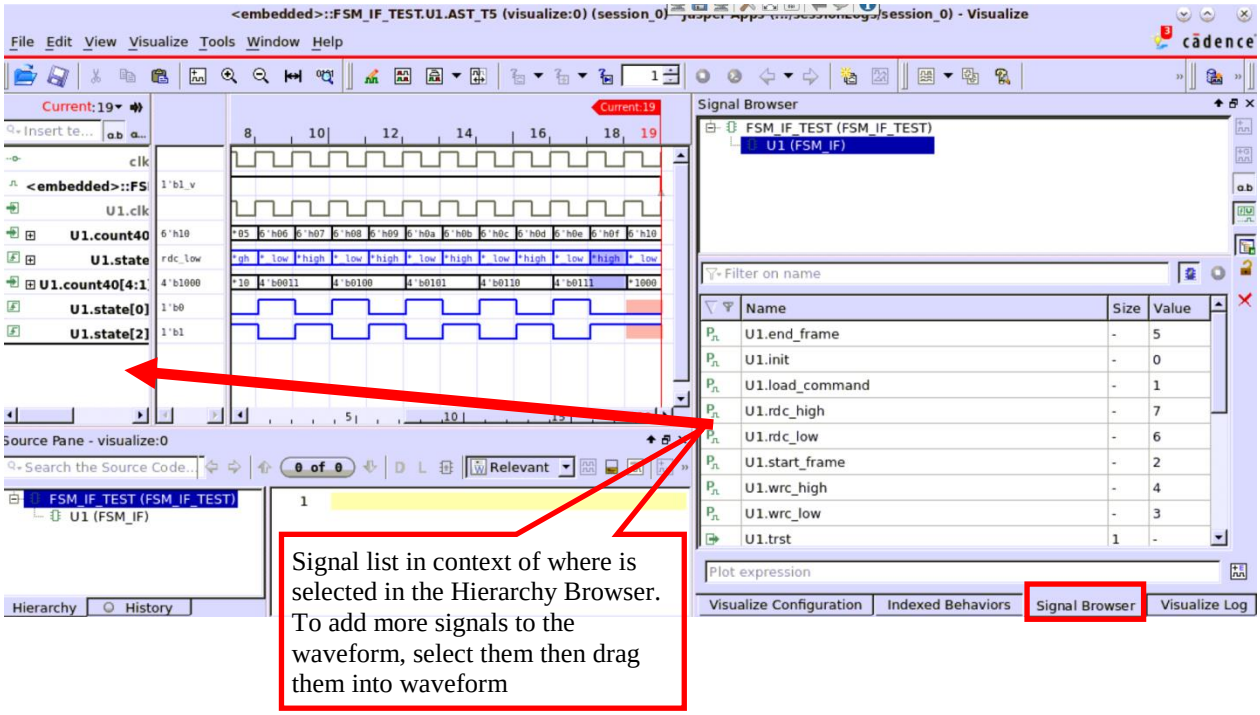
1. Visualize is the debugging window, and it's how we interact with Jasper graphically. Visualize allows us to observe waveforms (traces) of CEXs or observing covers passing.

2. To open visualize, in the property table, **double**-click a failing assertion (has a red cross) or a cover that is passing (has a green tick).

© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

When you double-click a CEX you see something like the diagram below. Only the signals referred to in the property are shown initially.

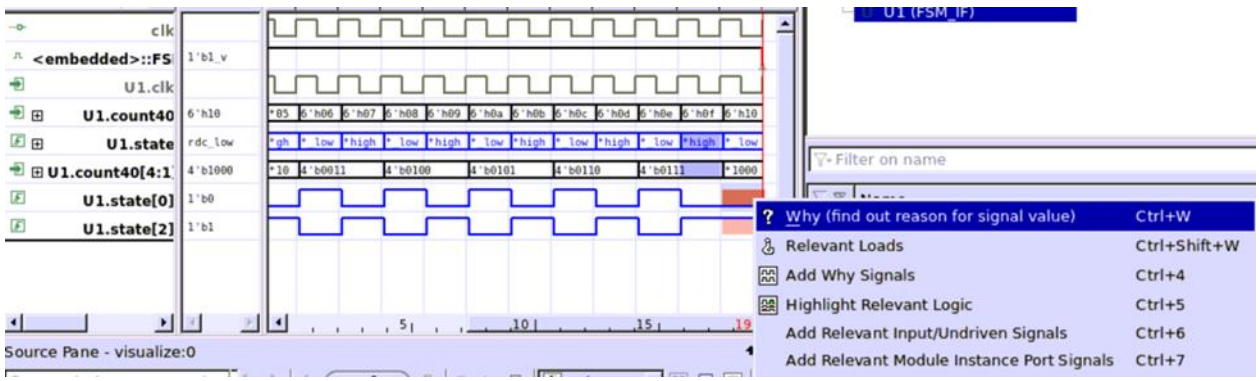


3. Notice that the signals `U1.state[*]` are highlighted in red in the final cycle where the assertion fails. Jasper is suggesting to you that it is likely that the value of those signals in those cycles are the reason for the failure.

4. Notice that the signals `U1.state[*]` and `U1.count40` are highlighted in blue in cycles before the assertion fails. Jasper is suggesting to you that it likely that the value of those signals in those cycles are the cause or precursor (antecedent) of the failure.

5. Debugging is just like looking at a waveform from a simulation.

6. You can also see the cause of any signal having its value on any cycle. To do this, select the signal you are interested in on the cycle you are interested in. Then click the RMB, and select Why?



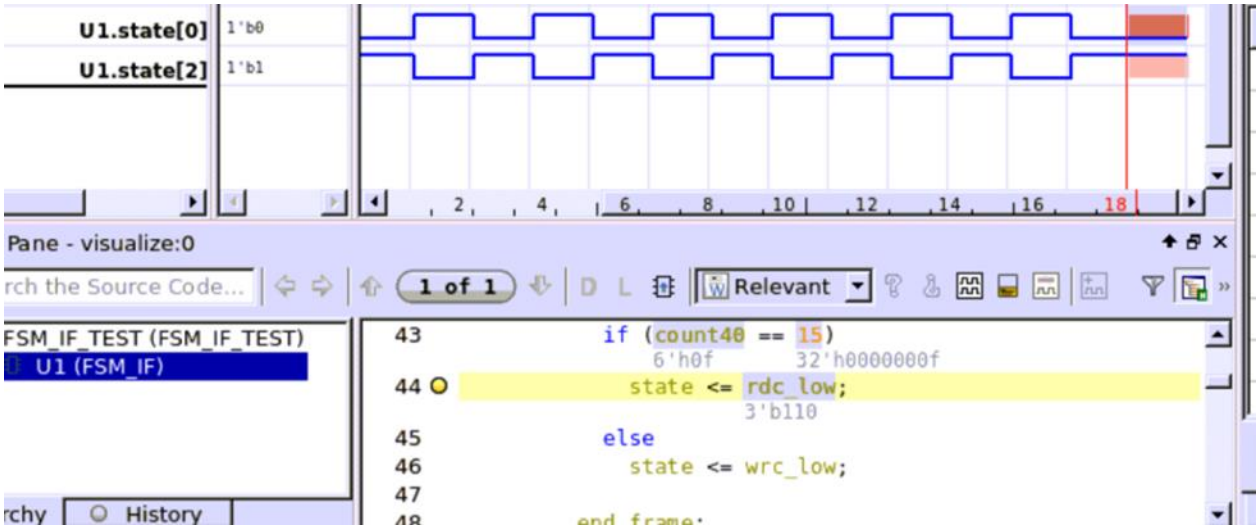
© 2022 Cadence Design Systems, Inc. All rights reserved.

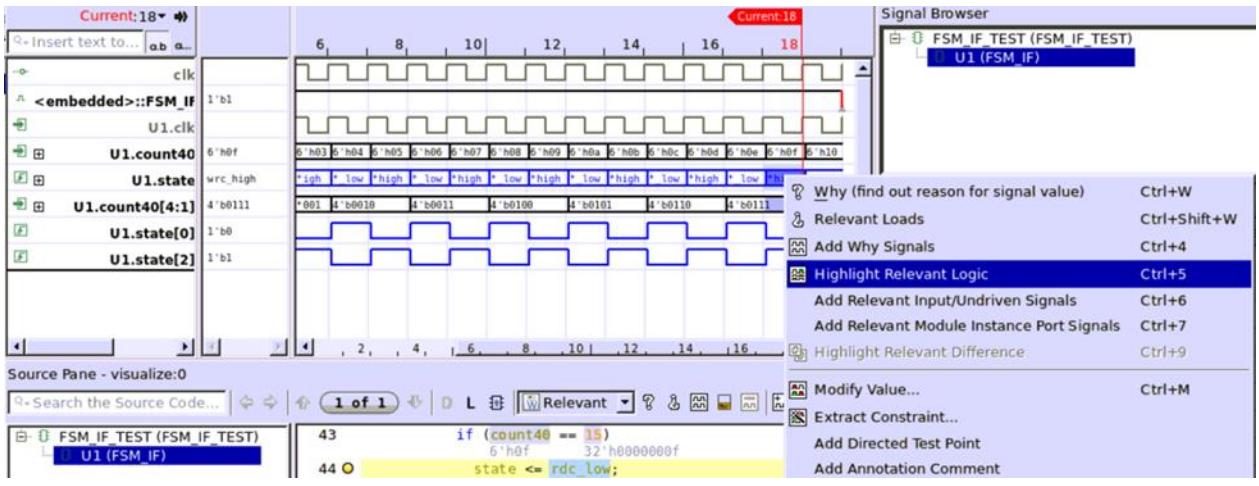
(c) Cadence Design Systems Inc. Do not distribute.

7. The source browser pane will show the assignment which gave that signal its value in that cycle you selected.

8. In the source browser, you can select any signal, click the RMB and select Why? Again, to follow back, which assignment gave that signal its value. With Jasper, we see an end result, the assertion failing, then we have to work backward and find out how we arrived at that point.

9. In visualize waveform window, you can select any signal, click the RMB and then select **Highlight Relevant Logic**. This will show all signals in the fanin cone of that selected signal going back over 2 FF boundaries by default. These fan-in signals will be added to the waveform.





End of Lab

© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Appendix B: Xcelium Simulator Tool Tips Guide

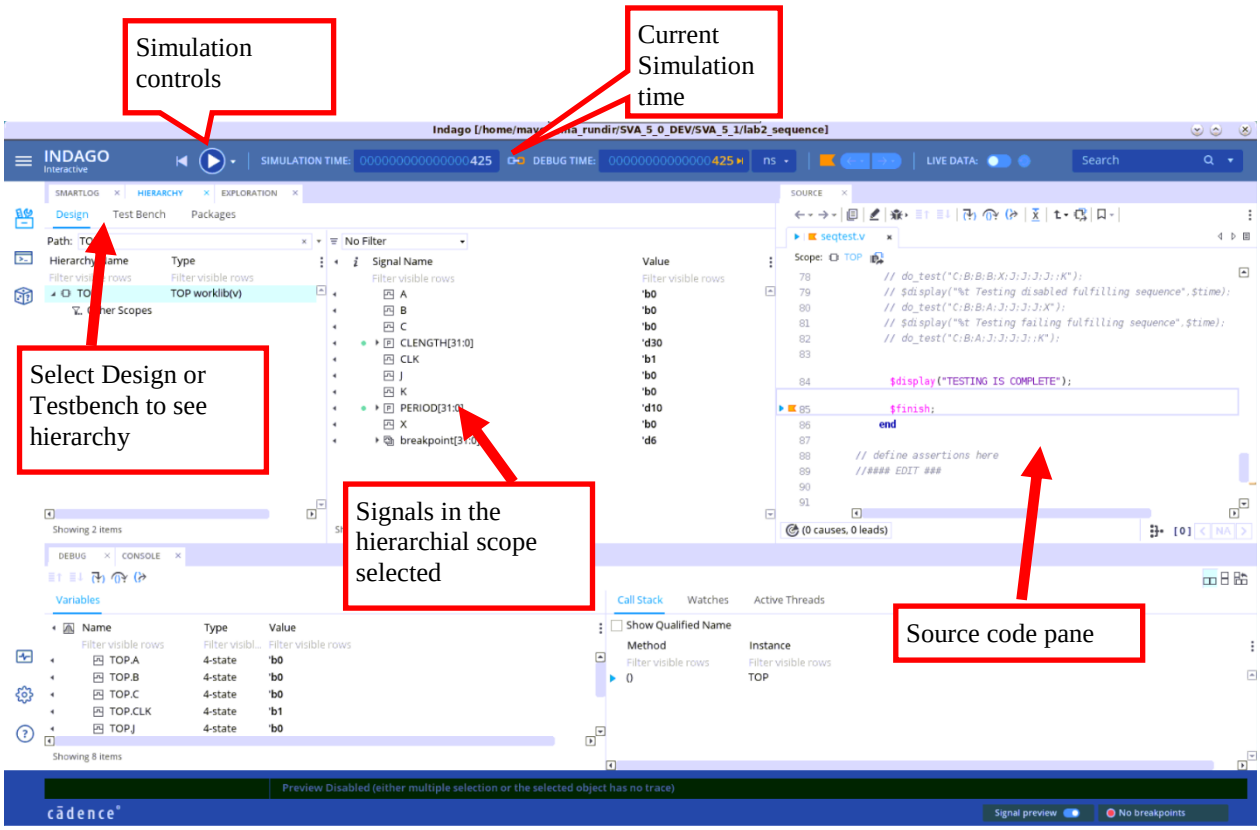
(c) Cadence Design Systems Inc. Do not distribute.

(c) Cadence Design Systems Inc. Do not distribute.

Introduction

This appendix is intended to give you a quick start to using the Xcelium™ Simulator specifically for debugging assertions and observing covers using the Indago™ GUI. The run scripts provided in the lab exercises will do all of the necessary tool setup and compilation tasks for you. Many scripts may also run the simulation for you. The Indago Debug GUI is an intuitive tool. Most of the time, if you hover your mouse over things, then it tells you what they are. If you select something and then click the right-hand mouse button (RMB), then a context-sensitive pop-up menu will make it clear what you can do from there.

Main Indago Window



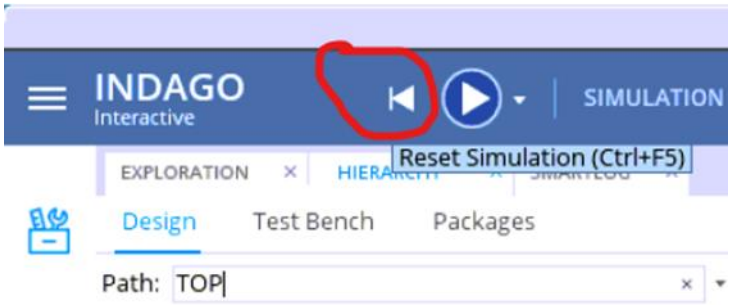
© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Setting Simulation Time Back to Zero

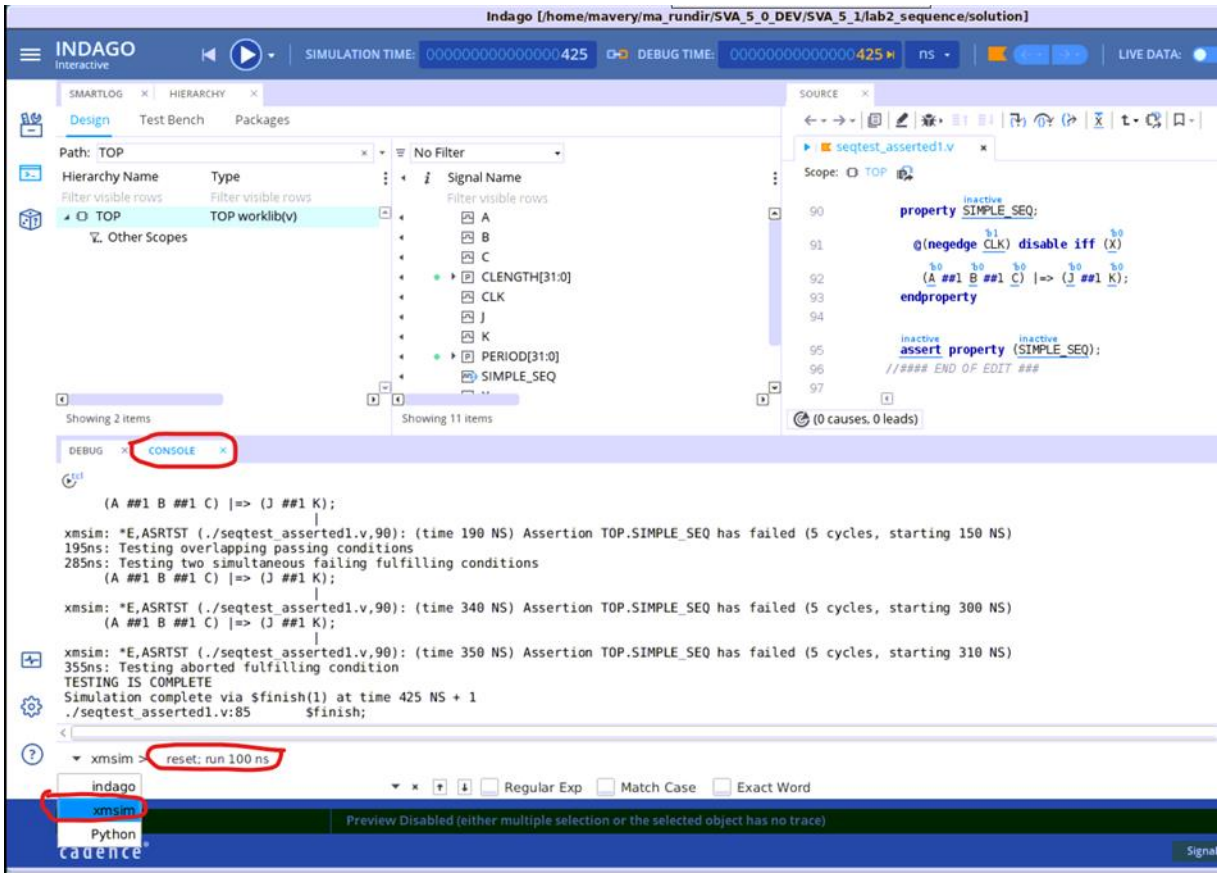
If your lab run script also runs the simulation, then the current simulation time will display a value that is not zero. Unless you explicitly and previously probed the signals and assertions that you wish to plot on a waveform, then you will have to set the simulation time back to zero. Data is only stored for plotting waveforms and assertions if you explicitly request it in advance of running the simulation.

1. Click the **Rewind** button in the simulation controls to set the current simulation time back to zero. You can then add the signals and assertions of interest to the waveform and then press the play button in the simulation controls to run the simulation again.



Command Console and Transcript

1. You can choose where commands entered at the command prompt get passed to. For simulator commands, choose the **xmsim** Tcl command prompt.



© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Plotting Signals as Waveforms

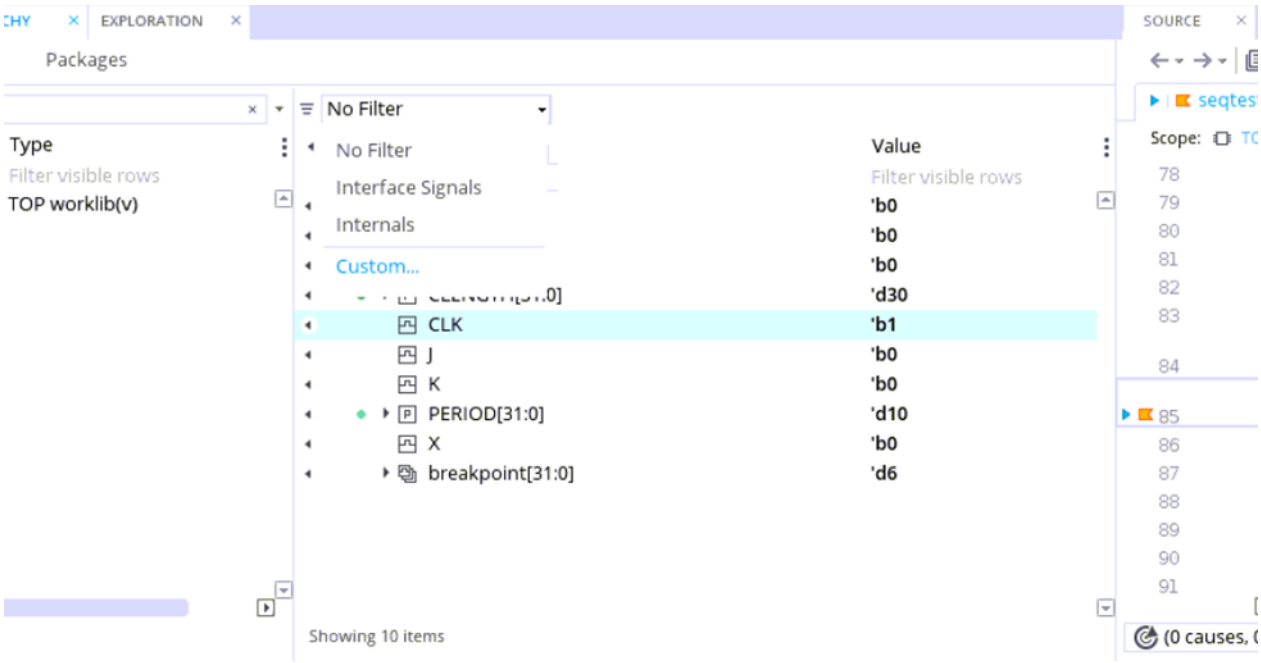
1. Select the signals(s) you wish to plot and then click the right-hand mouse button (RMB).

2. From the RMB pop-up menu, select **Send to Wave**. Note also the other options in the menu for future reference.

Filtering Objects in Scope

All Indago tables have a way of filtering in order to focus on what is important to you at that moment.

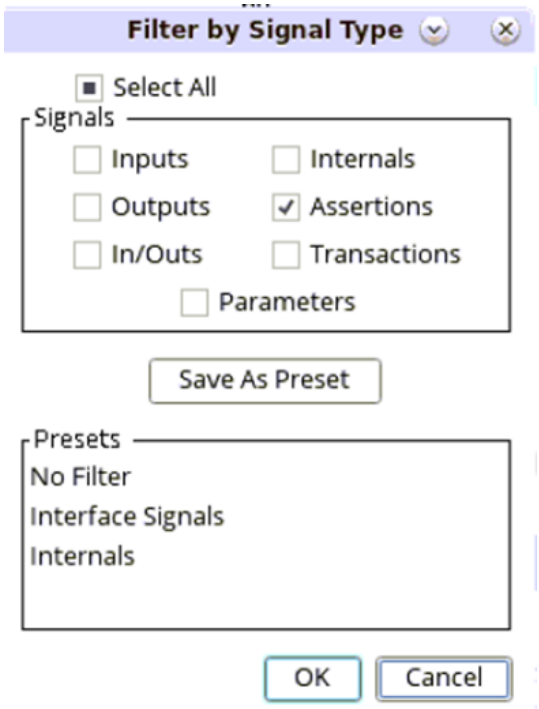
1. From the Filter drop-down menu, select **custom**.



© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

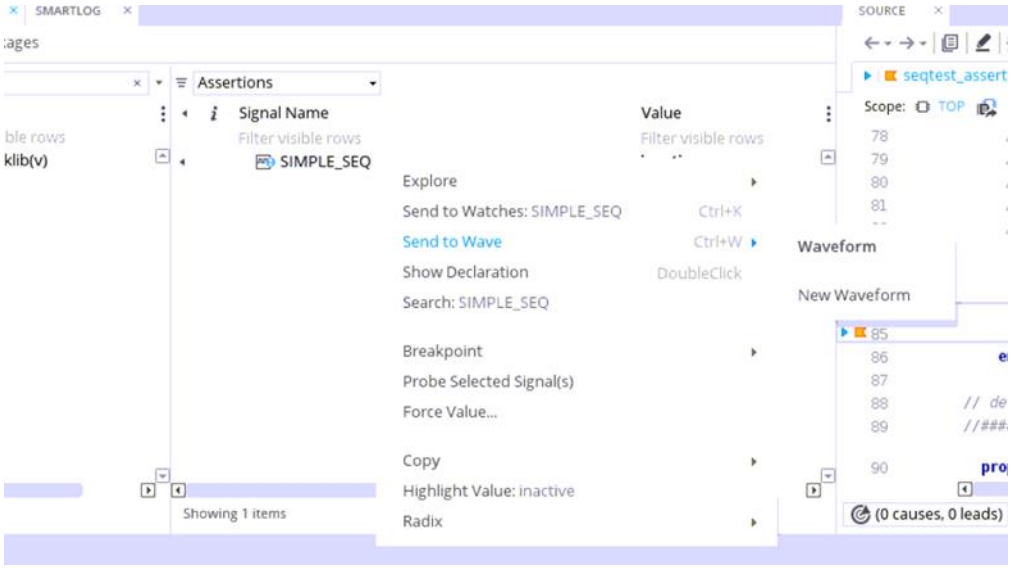
2. Select the kind of objects you wish to see, for example assertions only.



Plotting Assertions as Waveforms

1. Select the assertions(s) you wish to plot and then click the right-hand mouse button (RMB).

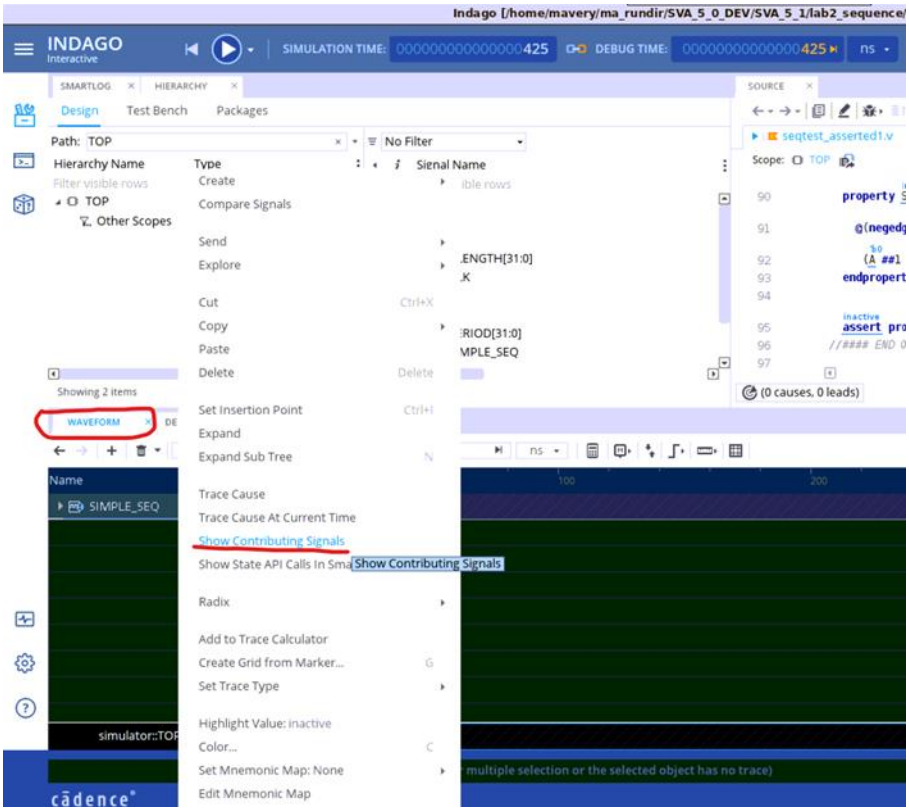
2. From the RMB pop-up menu, select **Send to Wave**.



© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

3. On the waveform window, select the assertion of interest, click the RMB and select **Show Contributing Signals**. This will automatically add to the waveform all signals to which the property refers to. Note also the other options in the menu for future reference.



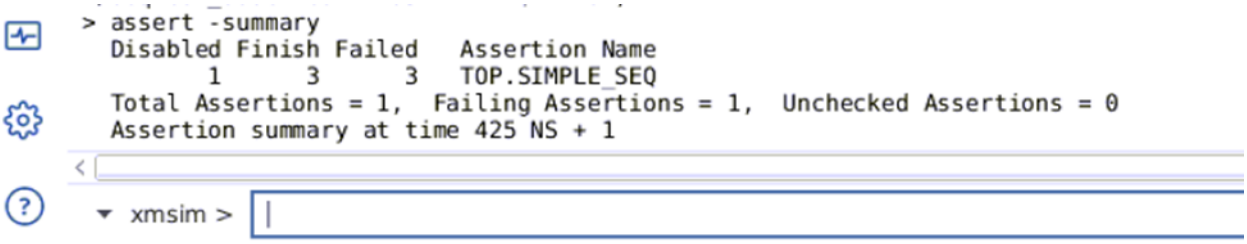
© 2022 Cadence Design Systems, Inc. All rights reserved.

(c) Cadence Design Systems Inc. Do not distribute.

Printing the Assertion Summary

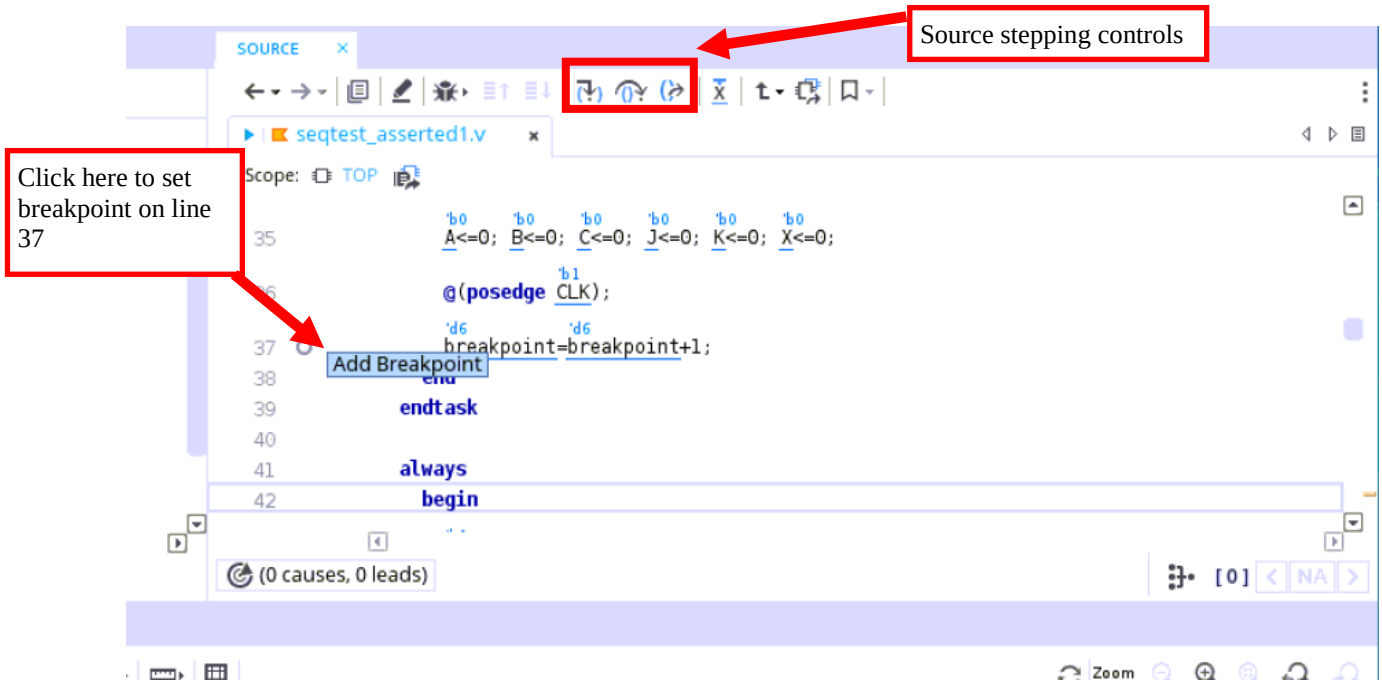
1. It is useful to see how the total number of covers, assertion passes and assertion failures we have had during simulation.

2. At the xmsim Tcl prompt, enter the command **assert -summary**.



Setting Breakpoints and Stepping Through Source Code

1. In the source window, click to the right of the line number which you wish to break on. A red dot will appear when the breakpoint is set. To disable the breakpoint, click again. To remove the breakpoint, click RMB and select remove from the pop-up menu.



2. Use the source stepping controls to step over, step into or step out of the current execution scope.

End of Lab

© 2022 Cadence Design Systems, Inc. All rights reserved.